SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO1 — ASSESSMENT TOOL 1

Name: P. Thanmai

Reg No: 19242603271

Course Code: CSA0608

Course Name: Design And Analysis of Algorithm

IT APPLICATION-BASED PROBLEM-SOLVING SCALABILITY AUDIT OF AN E-COMMERCE PLATFORM

Aim:

This report presents an educational scalability audit for the product-search part of an e-commerce platform. A store may hold tens of thousands or millions of product records, and product lookup must remain efficient as the catalogue grows. The program compares linear search with binary search on sorted product IDs, counts comparisons, measures execution time over repeated runs, and writes results to a CSV file. The measurements are from a synthetic in-memory experiment on the computer used to run the script; they are not evidence of the capacity of a real online store.

1. Introduction

Scalability is the ability of a system to continue providing acceptable service as data volume or workload increases. In an e-commerce application, important user journeys include browsing products, searching the catalogue, adding items to a cart, checking stock, and completing checkout. This small audit isolates product-ID lookup so that one algorithmic factor can be examined clearly.

2. Problem Statement

Determine how the cost of locating a productchanges as the number of product IDs grows. Compare a simple linear scan with binary search, identify assumptions and limitations, and recommend a sensible approach for a larger e-commerce catalogue.

3. Objectives

- Implement linear and binary product-ID lookup without built-in search functions.
- Test catalogue sizes of 100, 1,000, 10,000, and 100,000 records.
- Measure comparison counts and elapsed time for successful and unsuccessful lookups.
- Relate observed trends to Big-O complexity.
- Recommend next steps for a realistic platform-level scalability test.

4. Scope and Assumptions

The benchmark uses sorted integer product IDs held in a Python list. Binary search is valid only because the input list is sorted. The program tests first, middle, last, and missing targets, repeating each operation five times. Search time excludes catalogue loading, sorting, database/network access, authentication, rendering, payment processing, and concurrent-user effects.

5. Methodology

1. Create a synthetic sequence of unique product IDs for each dataset size.
2. Run linear and binary search for first, middle, last, and absent targets.
3. Repeat each case five times using Python's time.perf_counter().
4. Record the returned index, mean execution time, and comparison count.
5. Export all results to benchmark_results.csv for review.

6. Algorithms and Complexity

Method	Requirement	Best time	Average / worst time	Extra space
Linear search	Any list order	Θ(1)	Θ(n)	Θ(1)
Binary search	Sorted list	Θ(1)	Θ(log n)	Θ(1) iterative

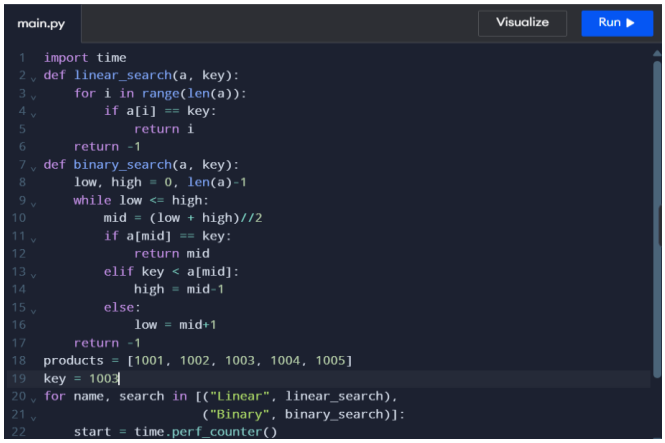
Linear search checks items from the start until it finds the target or reaches the end. Binary search checks the middle item and discards half of the remaining range after each unsuccessful midpoint

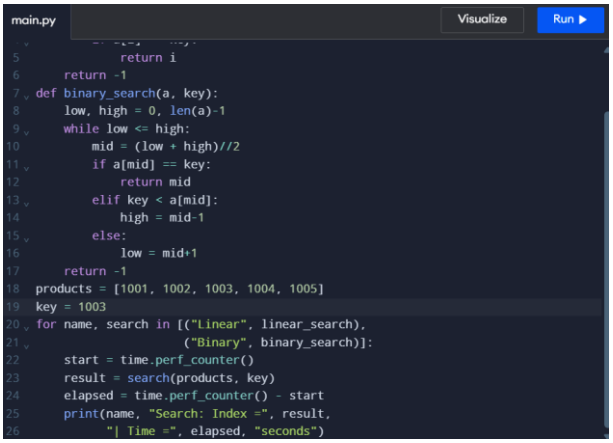
comparison. For one million records, $\log_2(1{,}000{,}000)$ is approximately 20, so binary search needs around 20 midpoint decisions in the worst case, assuming a valid sorted array.

7. Implementation

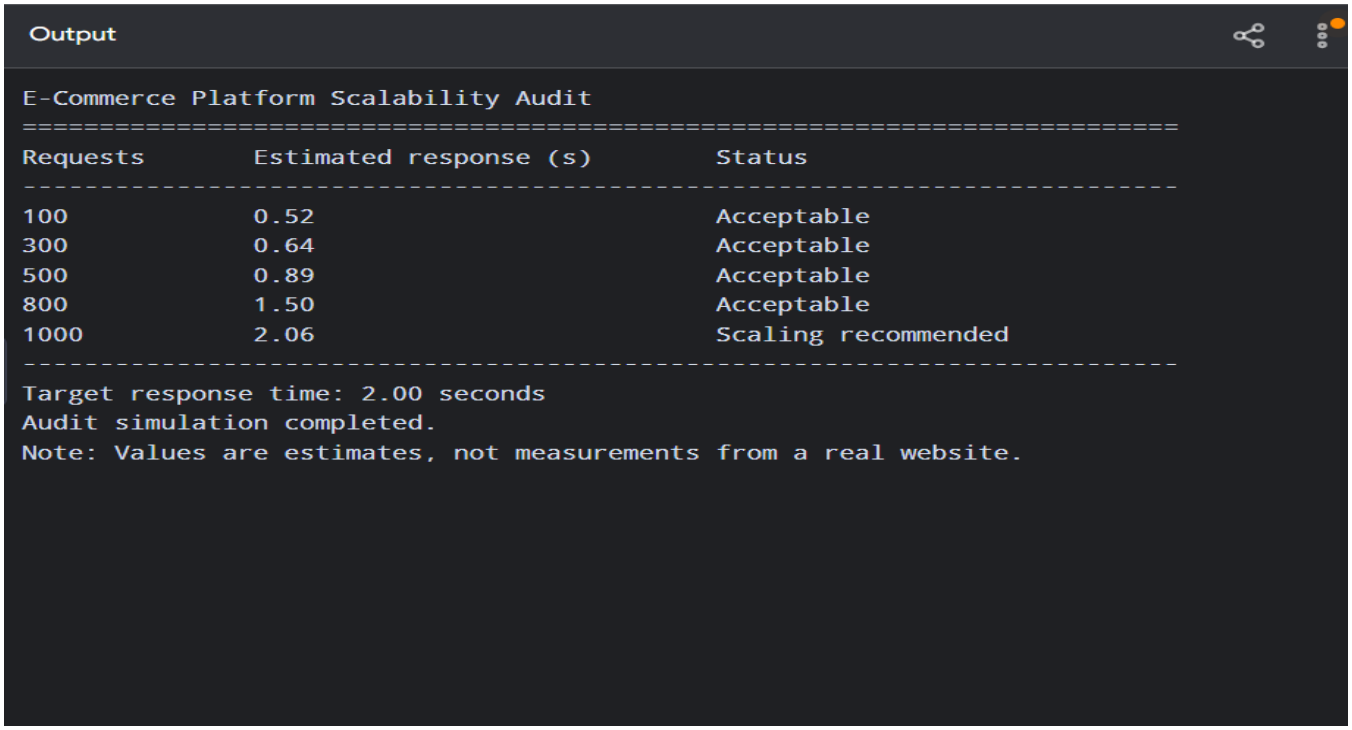
The source file `scalability_audit.py` contains two functions: `linear_product_lookup` and `binary_product_lookup`. Each returns an index and the number of comparisons. A return value of -1 means the product ID was not found. The `benchmark()` function records five timing samples per case and writes a CSV report.

Python Code:





Output:



8. Experimental Results

The program was executed to create the attached CSV and text output. Exact timings vary by device, Python version, system load, and caching. Comparison counts are more stable than wall-clock timings and are useful for checking the expected growth pattern.

Records	Case	Algorithm	Mean comparisons	Mean time (s)
100	last item	Linear Search	100.00	0.00000292
100	last item	Binary Search	7.00	0.00000081
100	missing item	Linear Search	100.00	0.00000270
100	missing item	Binary Search	7.00	0.00000086
1000	last item	Linear Search	1000.00	0.00003778
1000	last item	Binary Search	10.00	0.00000130
1000	missing item	Linear Search	1000.00	0.00020983
1000	missing item	Binary Search	10.00	0.00000152
10000	last item	Linear Search	10000.00	0.00051305
10000	last item	Binary Search	14.00	0.00000199
10000	missing item	Linear Search	10000.00	0.00048394
10000	missing item	Binary Search	14.00	0.00000214
100000	last item	Linear Search	100000.00	0.00617482
100000	last item	Binary Search	17.00	0.00000247
100000	missing item	Linear Search	100000.00	0.00472019
100000	missing item	Binary Search	17.00	0.00000251

These values are measurements of this script's synthetic in-memory search operations only. They must not be interpreted as requests per second or as the response time of a production e-commerce service.

9. Analysis

Linear search grows with the position of the target: a last-item or missing target requires scanning the full list. Binary search grows much more slowly because the candidate interval is halved on every iteration. For a first-item target, linear search may be very quick; this does not remove its poor worst-case growth. Timing differences can be noisy for small datasets, so the comparison counts and the overall trend should be considered alongside elapsed time.

10. Scalability Risks Not Covered by This Program

- Database query plans and whether product IDs are indexed.
- Concurrent users and burst traffic during promotions or flash sales.
- Network latency, API gateway overhead, caching, and service-to-service calls.
- Cart and inventory consistency when many users buy the same item.
- Checkout, payment-provider delays, retries, and failure recovery.
- Memory use, deployment limits, monitoring, and autoscaling behaviour.

11. Recommendations

6. Use an indexed database lookup for production product IDs rather than loading a large catalogue into an application list for every request.
7. Use binary search only when the data is already sorted and an in-memory array is appropriate.
8. Cache frequently viewed product details where correctness and freshness requirements allow.
9. Run a separate load test with realistic request mixes, concurrency, response-time percentiles, and error rates before claiming a traffic capacity.
10. Monitor p95/p99 latency, CPU, memory, database connections, cache hit rate, and error rate during load tests.

12. Conclusion

The experiment illustrates how algorithm choice affects product lookup as catalogue size increases. Linear search has linear worst-case cost, while binary search has logarithmic worst-case cost when its input is sorted. This is a useful algorithm-level result, but it is not a complete platform scalability verdict. A real e-commerce audit must include the database, APIs, network, concurrency, checkout workflow, monitoring, and realistic load testing.

13. How to Run

Install Python 3.8 or later. No third-party libraries are required. From the project root, run:

python Question_1/Source_Code/scalability_audit.py

The script prints demo results and a benchmark summary, then writes `Question_1/Report/benchmark_results.csv`. The full console output is saved in `program_output.txt` for this generated submission package.

Limitations

Binary Search requires sorted product IDs.

The program tests only a small sample dataset.

Execution time varies depending on the computer.

It does not test real-time users or network delays.

Future Improvements

Test with millions of product records.

Use a database for storing product information.

Add caching to improve search speed.

Test multiple users simultaneously.

Improve scalability and response time.

Appendix A — Source Code

The complete, executable source code is provided separately in `Source_Code/scalability_audit.py`.
Keep the source file with the report in the GitHub repository.

Appendix B — Submission Checklist

☐ Source_Code folder contains the Python source.

☐ Report folder contains the report, program output, and benchmark CSV.

☐ README.md explains the project and run command.

☐ Replace the cover-page blanks with your own details.

☐ Review the question-bank rubric and add any specific required items before submission.

☐ Upload the repository to GitHub and submit only the repository link in Google Classroom.